LAB ASSIGNMENT – 11.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

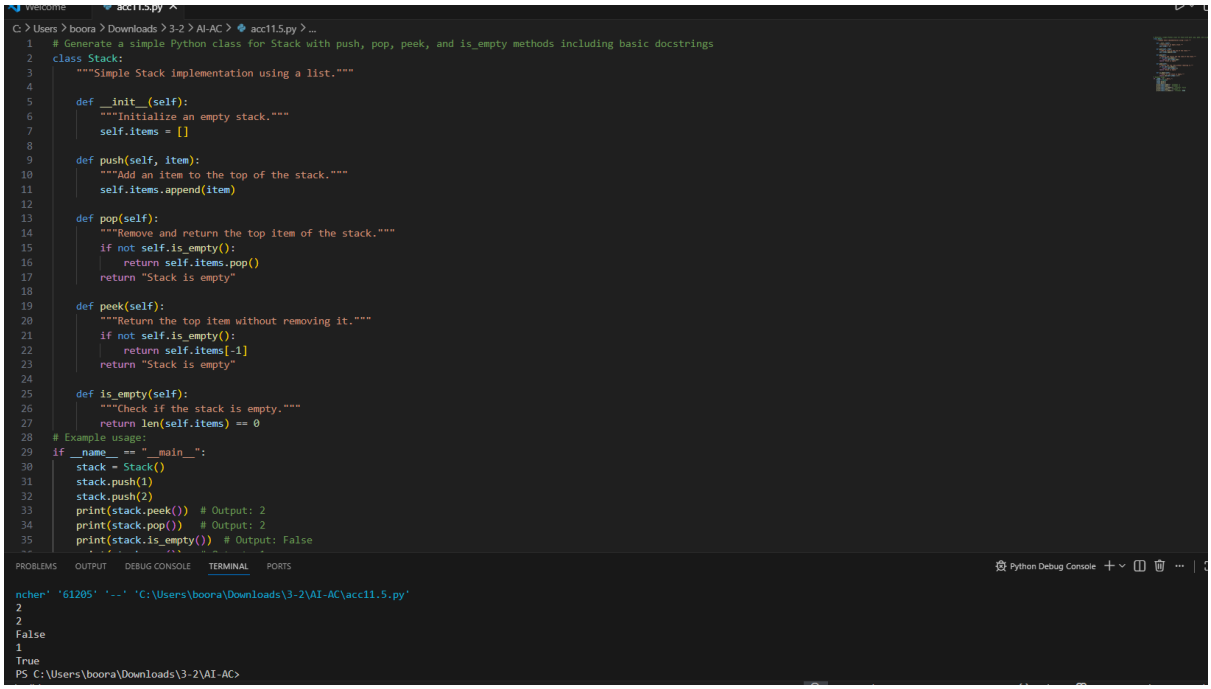
• A functional stack implementation with all required methods and docstrings.

Code:

```
# Generate a simple Python class for Stack with push, pop, peek, and is_empty methods including basic docstrings
class Stack:
    """Simple Stack implementation using a list."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)
```

```
    def pop(self):
        """Remove and return the top item of the stack."""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return the top item without removing it."""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    print(stack.peek())  # Output: 2
    print(stack.pop())   # Output: 2
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

Output:



Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size

methods.

Code:

Generate a simple Python Queue class using lists with enqueue, dequeue, peek, and size methods including basic docstrings.

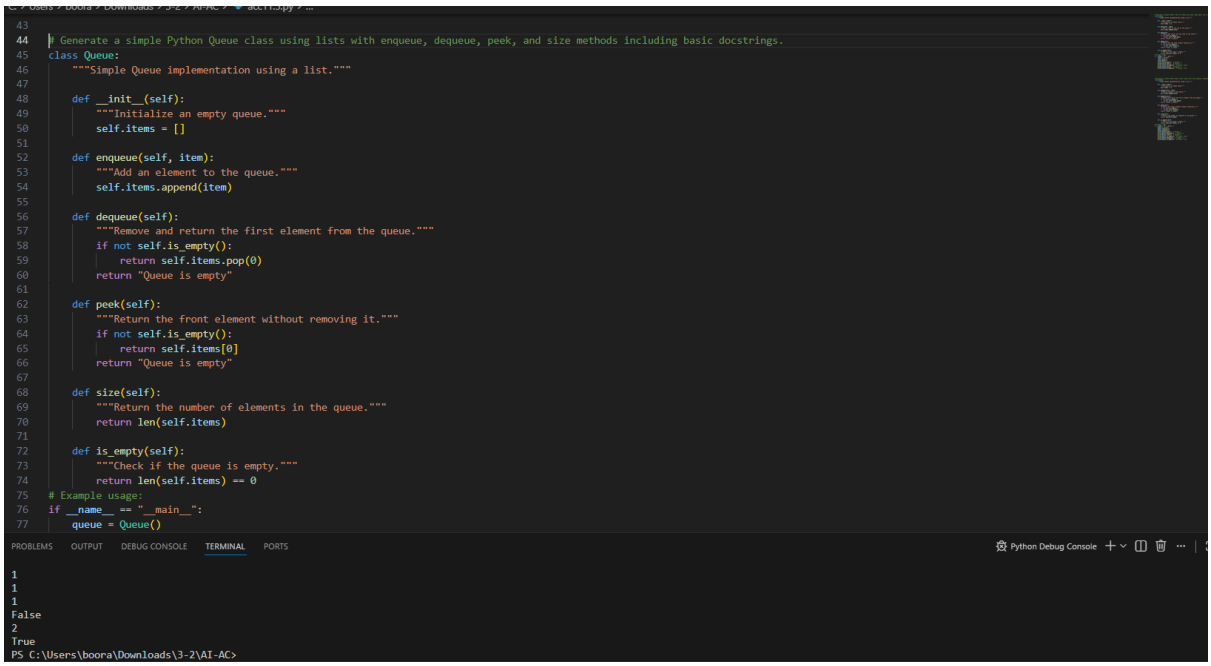
class Queue:

"""Simple Queue implementation using a list."""

```
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an element to the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the first element from the queue."""
        if not self.is_empty():
            return self.items.pop(0)
        return "Queue is empty"

    def peek(self):
        """Return the front element without removing it."""
        if not self.is_empty():
            return self.items[0]
        return "Queue is empty"

    def size(self):
        """Return the number of elements in the queue."""
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0
# Example usage:
```

```
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    print(queue.peek())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.size())  # Output: 1
    print(queue.is_empty())  # Output: False
    print(queue.dequeue())  # Output: 2
    print(queue.is_empty())  # Output: True
```

Output:



Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method

documentation.

Code:

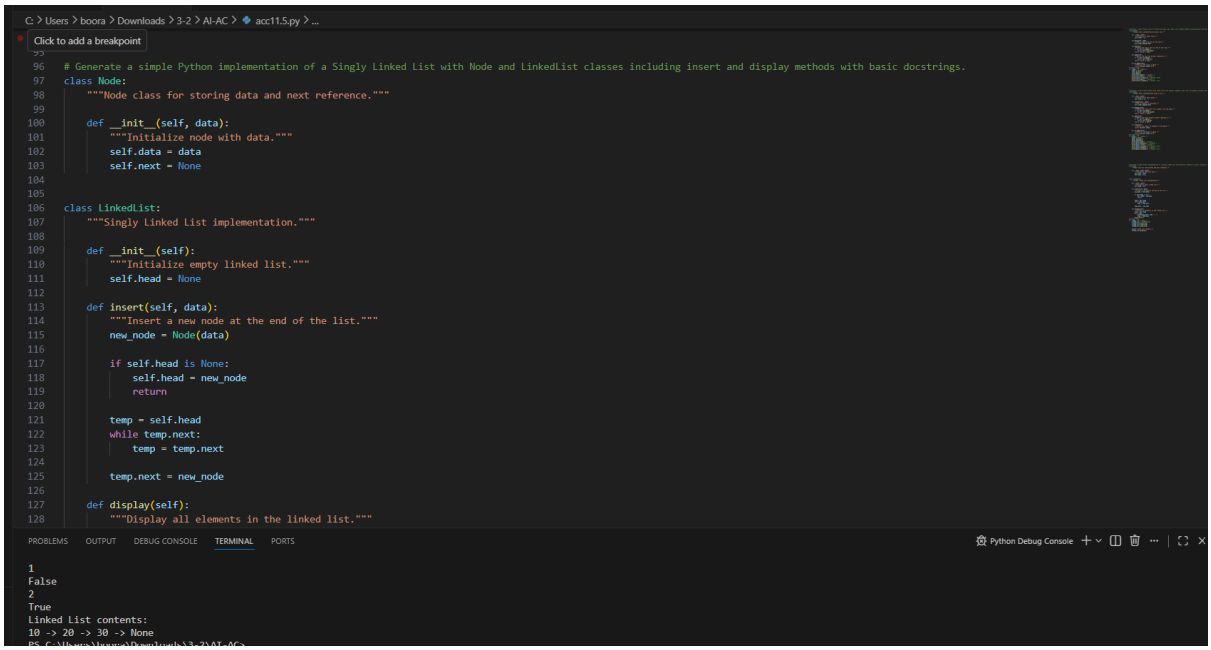
Generate a simple Python implementation of a Singly Linked List with Node and LinkedList classes including insert and display methods with basic docstrings.

```
class Node:
    """Node class for storing data and next reference."""

    def __init__(self, data):
        """Initialize node with data."""
        self.data = data
        self.next = None


class LinkedList:
    """Singly Linked List implementation."""

    def __init__(self):
        """Initialize empty linked list."""
        self.head = None

    def insert(self, data):
```

```
        """Insert a new node at the end of the list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    def display(self):
        """Display all elements in the linked list."""
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
# Example usage
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)

    print("Linked List contents:")
```

linked_list.display()

Output:



Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

Generate a simple Python HashTable class using chaining for collision handling with insert, search, and delete methods and basic docstrings.

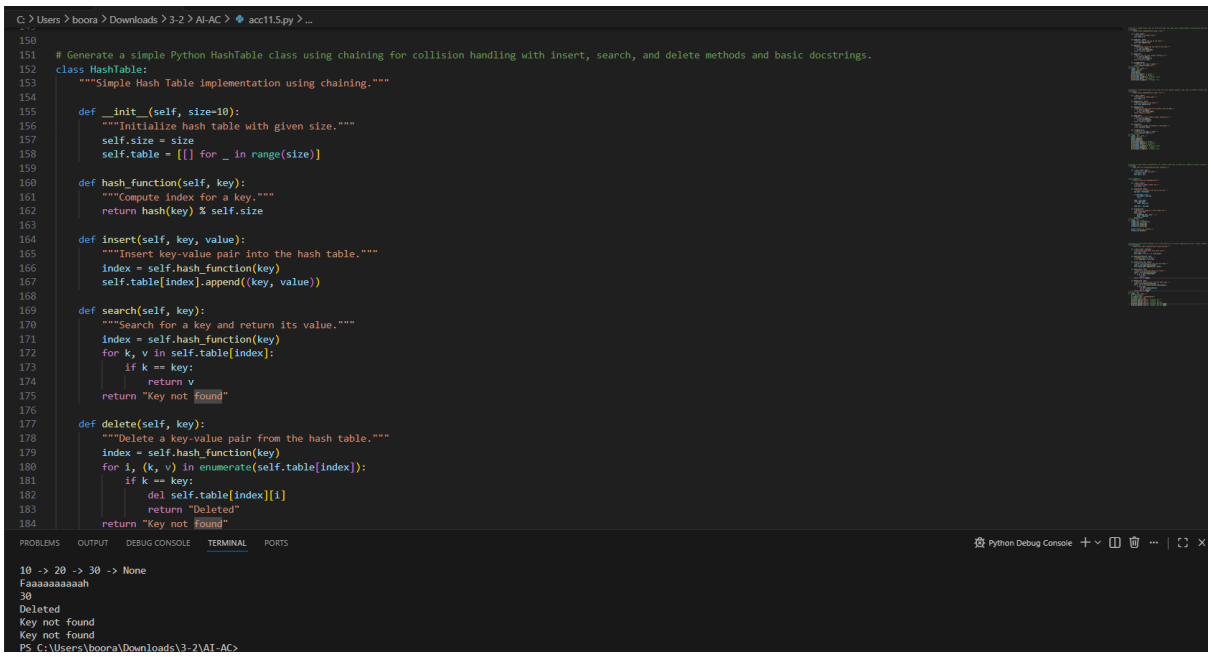
class HashTable:

"""Simple Hash Table implementation using chaining."""

```
def __init__(self, size=10):
    """Initialize hash table with given size."""
    self.size = size
    self.table = [[] for _ in range(size)]

def hash_function(self, key):
    """Compute index for a key."""
    return hash(key) % self.size

def insert(self, key, value):
    """Insert key-value pair into the hash table."""
    index = self.hash_function(key)
    self.table[index].append((key, value))

def search(self, key):
    """Search for a key and return its value."""
    index = self.hash_function(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return "Key not found"

def delete(self, key):
    """Delete a key-value pair from the hash table."""
    index = self.hash_function(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
```

```
            del self.table[index][i]
            return "Deleted"
        return "Key not found"
# Example usage
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("name", "Faaaaaaaaaah")
    ht.insert("age", 30)
    print(ht.search("name"))  # Output: Alice
    print(ht.search("age"))   # Output: 30
    print(ht.delete("name"))  # Output: Deleted
    print(ht.search("name"))  # Output: Key not found
    print(ht.delete("name"))  # Output: Key not found
```

Output:



Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections.

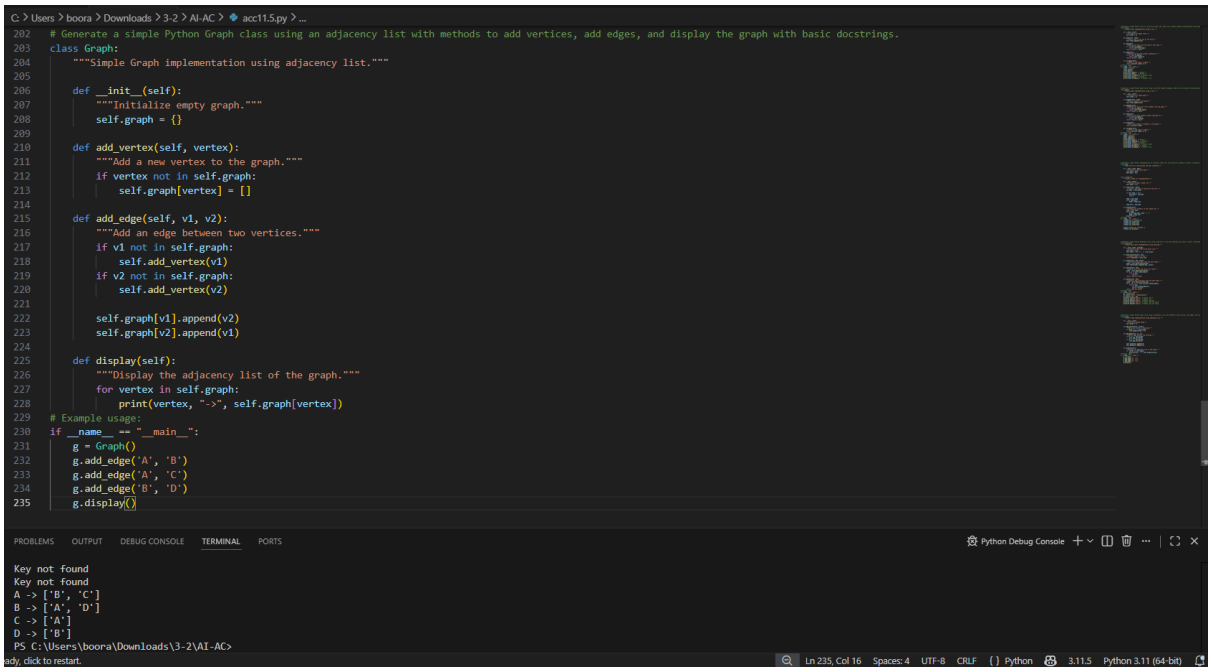
Code:

Generate a simple Python Graph class using an adjacency list with methods to add vertices, add edges, and display the graph with basic docstrings.

```
class Graph:
    """Simple Graph implementation using adjacency list."""

    def __init__(self):
        """Initialize empty graph."""
        self.graph = {}

    def add_vertex(self, vertex):
        """Add a new vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, v1, v2):
        """Add an edge between two vertices."""
        if v1 not in self.graph:
            self.add_vertex(v1)
```

```
        if v2 not in self.graph:
            self.add_vertex(v2)

        self.graph[v1].append(v2)
        self.graph[v2].append(v1)

    def display(self):
        """Display the adjacency list of the graph."""
        for vertex in self.graph:
            print(vertex, "->", self.graph[vertex])
# Example usage:
if __name__ == "__main__":
    g = Graph()
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.display()
```

Output:



Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

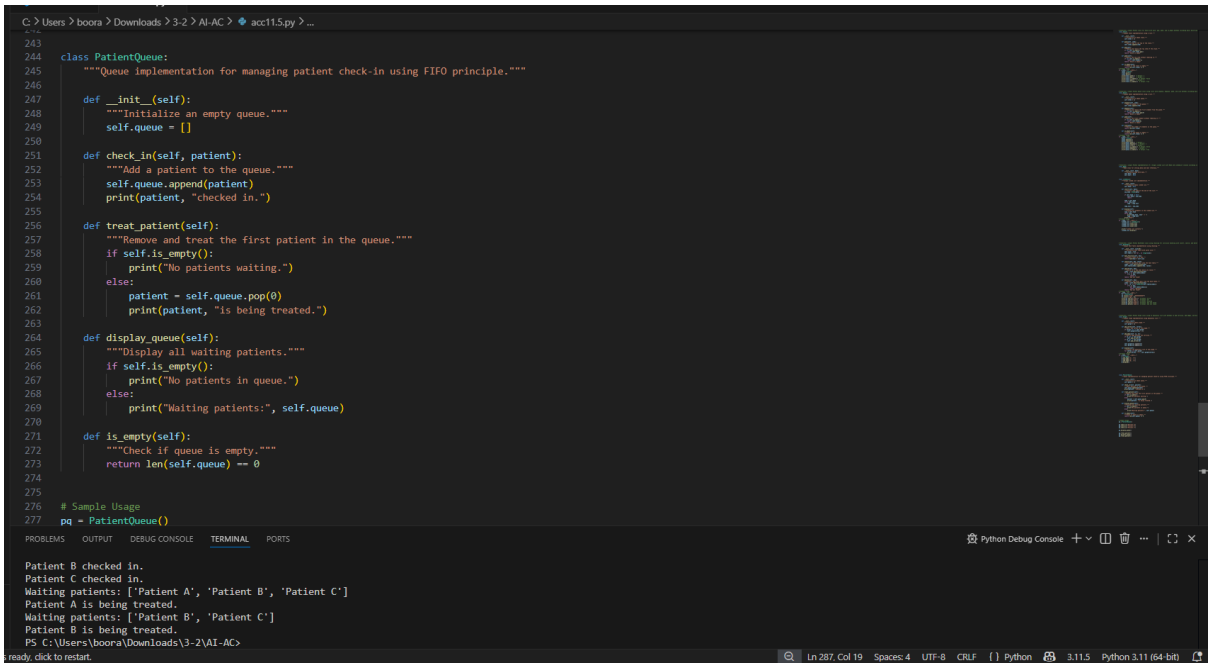
with comments and docstrings.

Code:

```
class PatientQueue:
    """Queue implementation for managing patient check-in using FIFO principle."""

    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def check_in(self, patient):
        """Add a patient to the queue."""
        self.queue.append(patient)
        print(patient, "checked in.")

    def treat_patient(self):
        """Remove and treat the first patient in the queue."""
        if self.is_empty():
            print("No patients waiting.")
        else:
            patient = self.queue.pop(0)
            print(patient, "is being treated.")

    def display_queue(self):
        """Display all waiting patients."""
        if self.is_empty():
            print("No patients in queue.")
```

```
        else:
            print("Waiting patients:", self.queue)

    def is_empty(self):
        """Check if queue is empty."""
        return len(self.queue) == 0



# Sample Usage
pq = PatientQueue()

pq.check_in("Patient A")
pq.check_in("Patient B")
pq.check_in("Patient C")

pq.display_queue()

pq.treat_patient()
pq.display_queue()
pq.treat_patient()
```

Output:



Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings. Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks

prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected

logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

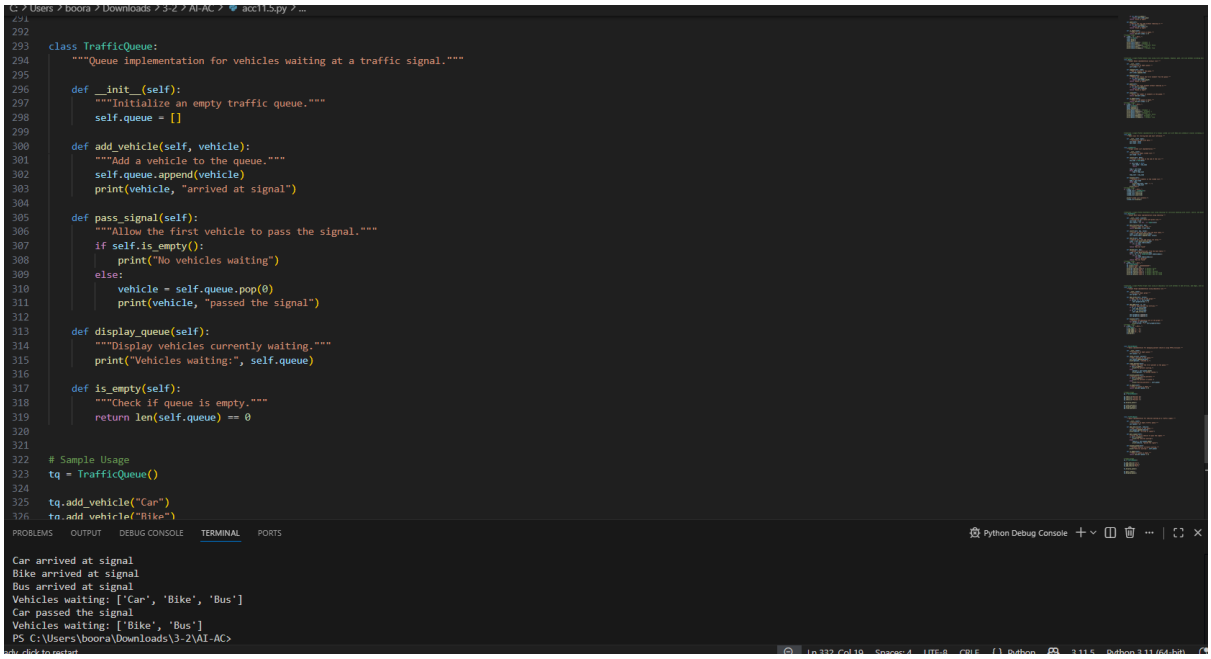
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
class TrafficQueue:
    """Queue implementation for vehicles waiting at a traffic signal."""

    def __init__(self):
        """Initialize an empty traffic queue."""
        self.queue = []

    def add_vehicle(self, vehicle):
        """Add a vehicle to the queue."""
        self.queue.append(vehicle)
        print(vehicle, "arrived at signal")

    def pass_signal(self):
        """Allow the first vehicle to pass the signal."""
        if self.is_empty():
            print("No vehicles waiting")
        else:
```

```
            vehicle = self.queue.pop(0)
            print(vehicle, "passed the signal")

    def display_queue(self):
        """Display vehicles currently waiting."""
        print("Vehicles waiting:", self.queue)

    def is_empty(self):
        """Check if queue is empty."""
        return len(self.queue) == 0


# Sample Usage
tq = TrafficQueue()

tq.add_vehicle("Car")
tq.add_vehicle("Bike")
tq.add_vehicle("Bus")

tq.display_queue()

tq.pass_signal()
tq.display_queue()
```

Output:



Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

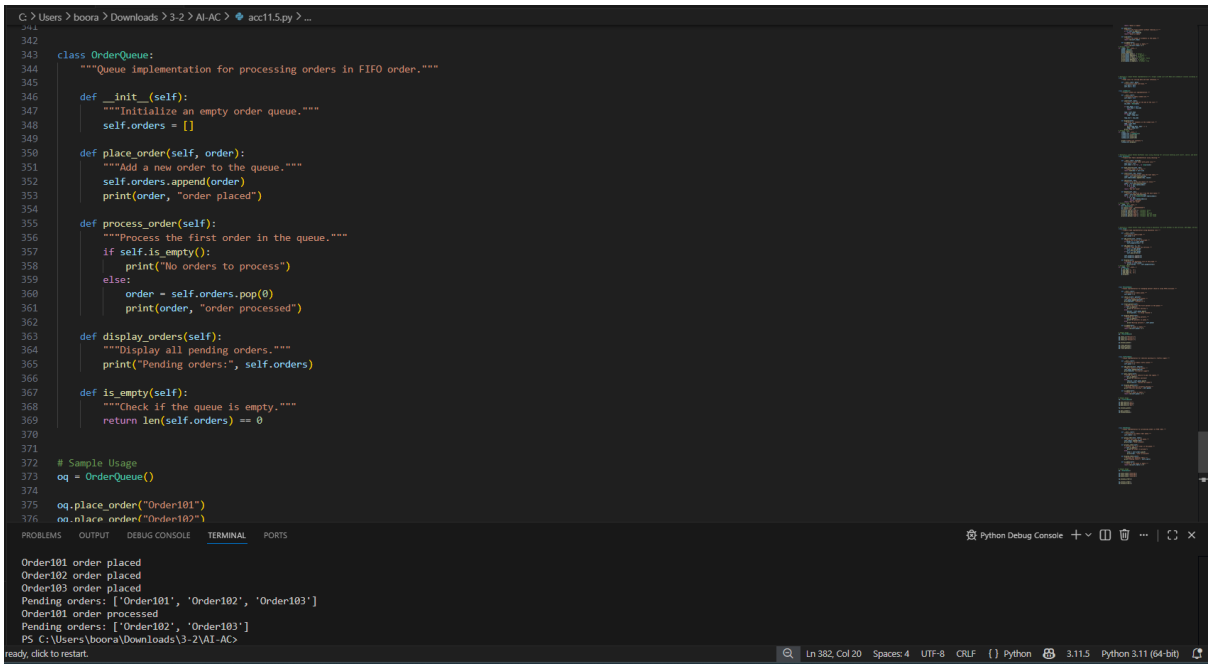
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
class OrderQueue:
    """Queue implementation for processing orders in FIFO order."""

    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = []

    def place_order(self, order):
        """Add a new order to the queue."""
        self.orders.append(order)
        print(order, "order placed")
```

```
    def process_order(self):
        """Process the first order in the queue."""
        if self.is_empty():
            print("No orders to process")
        else:
            order = self.orders.pop(0)
            print(order, "order processed")

    def display_orders(self):
        """Display all pending orders."""
        print("Pending orders:", self.orders)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.orders) == 0


# Sample Usage
oq = OrderQueue()

oq.place_order("Order101")
oq.place_order("Order102")
oq.place_order("Order103")

oq.display_orders()

oq.process_order()
oq.display_orders()
```

Output:



Justifications:

Task 1 – Stack Implementation (Justification)

Stack follows **LIFO (Last In First Out)** principle.

The last inserted element is removed first.

It is useful for operations like undo, recursion, and expression evaluation.

Task 2 – Queue Implementation (Justification)

Queue follows **FIFO (First In First Out)** principle.

Elements are processed in the same order they arrive.

It is commonly used in scheduling and task management systems.

Task 3 – Linked List (Justification)

Linked List stores elements using nodes connected by links.

It allows **dynamic memory allocation**.

Insertion and deletion operations are efficient compared to arrays.

Task 4 – Hash Table (Justification)

Hash Table stores data using **key-value pairs**.

It uses a hash function to quickly locate data.

Searching, insertion, and deletion operations are very fast.

Task 5 – Graph Representation (Justification)

Graph represents relationships between objects.

Nodes represent entities and edges represent connections.

It is useful for networks like roads, social networks, and maps.

Task 6 – Smart Hospital Management System (Justification)

Different hospital operations require different data structures.

Queue manages patient arrival order, while Priority Queue handles emergency cases.

Hash Table, BST, and Graph help manage records, appointments, and navigation efficiently.

Task 7 – Smart City Traffic Control System (Justification)

Traffic systems require organized vehicle and road management.

Queue manages vehicles at signals and Priority Queue handles emergency vehicles.

Hash Table and Graph help manage vehicle data and road network connections.

Task 8 – Smart E-Commerce Platform (Justification)

E-commerce platforms manage orders, products, and delivery routes.

Queue processes orders sequentially while Priority Queue tracks top-selling products.

Hash Table and Graph help in fast product search and delivery route planning.